

High-Level Design (HLD) Document

Project Overview

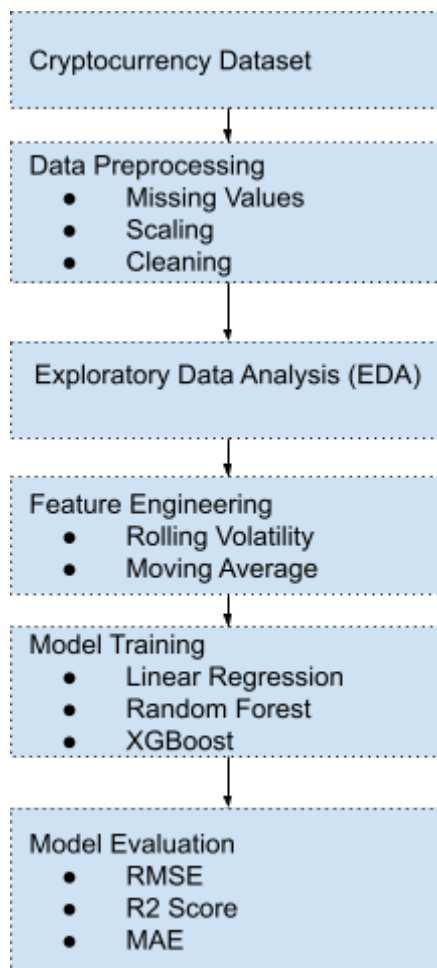
The system predicts cryptocurrency volatility using historical market data and machine learning techniques.

Objective

To develop a machine learning pipeline capable of:

- Processing cryptocurrency data
- Engineering predictive features
- Predicting future volatility
- Displaying predictions through a user interface

System Architecture



Modules Description

1. Data Collection Module

Responsible for loading cryptocurrency historical data from CSV files or APIs.

2. Data Preprocessing Module

Handles:

- Missing value treatment
- Duplicate removal
- Data formatting
- Feature scaling and normalization

3. EDA Module

Performs:

- Trend analysis
- Correlation analysis

4. Feature Engineering Module

Creates advanced features:

- Rolling volatility
- Moving averages
- Liquidity ratio
- Bollinger Bands

5. Model Training Module

Implements machine learning algorithms:

- Linear Regression
- Random Forest Regressor
- XGBoost Regressor

6. Evaluation Module

Measures model performance using:

- RMSE
- MAE
- R^2 Score

Technology Stack

Component	Technology
Programming Language	Python
Notebook Environment	Google Colab
Data Processing	Pandas, NumPy
Visualization	Matplotlib, Seaborn, Plotly
Machine Learning	Scikit-learn, XGBoost

Functional Requirements

- Import cryptocurrency historical dataset
- Clean and preprocess data
- Generate technical indicators
- Train ML models
- Predict volatility values
- Compare model performances

Non-Functional Requirements

- High prediction accuracy
- Fast prediction response
- Scalable architecture
- Easy maintainability
- User-friendly interface

2. LOW LEVEL DESIGN (LLD)

Detailed Workflow

Dataset Loading

|

v

Data Cleaning

|

v

Feature Engineering

|

v

Train-Test Split

|

v

Model Training

|

v

Prediction

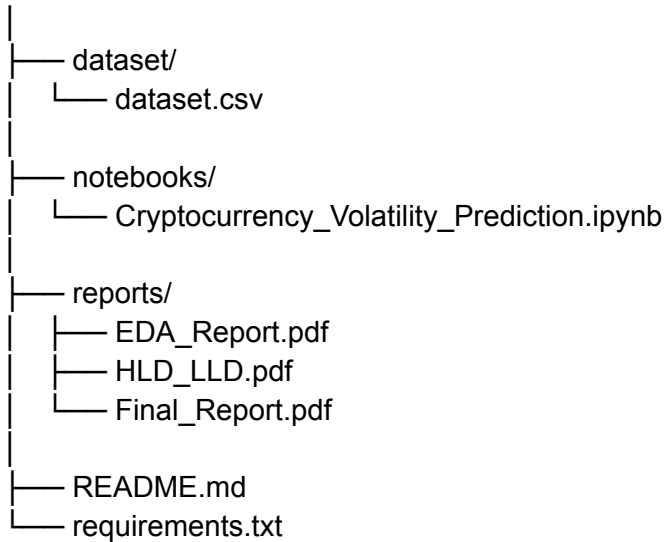
|

v

Evaluation Metrics

Folder Structure

Cryptocurrency-Volatility-Prediction/



Component Design

A. preprocessing.py

Responsibilities

- Handle missing values
- Convert date columns
- Normalize numerical features
- Remove duplicate entries

Functions

load_data()

clean_data()
normalize_features()

B. feature_engineering.py

Responsibilities

- Generate technical indicators
- Create rolling statistics
- Generate volatility metrics

Functions

calculate_returns()
calculate_volatility()
create_moving_averages()

C. train_model.py

Responsibilities

- Split dataset
- Train ML models
- Save trained models

Functions

train_linear_regression()
train_random_forest()
train_xgboost()
save_model()

D. evaluate.py

Responsibilities

- Generate evaluation metrics
- Compare models

Functions

calculate_rmse()

calculate_mae()
calculate_r2()

E. predict.py

Responsibilities

- Load trained model
- Generate predictions

Functions

load_model()
predict_volatility()

Database Design

Dataset Columns

Column Name	Description
Date	Trading date
Symbol	Cryptocurrency symbol
Open	Opening price
High	Highest price

Low	Lowest price
Close	Closing price
Volume	Trading volume
Market Cap	Total market capitalization
Volatility	Target variable